Fences are global operations and affect the ordering of other atomic operations in the thread that executed the fence. Fences are also commonly called *memory barriers*, and they get their name because they put a line in the code that certain operations can't cross. As you may recall from section 5.3.3, relaxed operations on separate variables can usually be freely reordered by the compiler or the hardware. Fences restrict this freedom and introduce happens-before and synchronizes-with relationships that weren't present before.

Let's start by adding a fence between the two atomic operations on each thread in listing 5.5, as shown in the following listing.

Listing 5.12. Relaxed operations can be ordered with fences

```
#include <atomic>
#include <thread>
#include <assert.h>

std::atomic<bool> x,y;
std::atomic<int> z;

void write_x_then_y()
{
    x.store(true,std::memory_order_relaxed);    <-1
    std::atomic_thread_fence(std::memory_order_release);      <-2
    y.store(true,std::memory_order_relaxed);            <-3
}

void read_y_then_x()
{
    while(!y.load(std::memory_order_relaxed));          <-4

    std::atomic_thread_fence(std::memory_order_acquire);      <-5
    if(x.load(std::memory_order_relaxed))               <-6
        ++z;
}

int main()
{
    x=false;
    y=false;
    z=0;
    std::thread a(write_x_then_y);
    std::thread b(read_y_then_x);
    a.join();
    b.join();
    assert(z.load()!=0);    <-7
}
```

The release fence ❷ synchronizes-with the acquire fence ❺, because the load from `y` at ❹ reads the value stored at ❸. This means that the store to `x` at ❶ happens-before the load from `x` at ❻, so the value read must be `true` and the assert at ❼ won't fire. This is in contrast to the original case without the fences where the store to and load from `x` weren't ordered, and so the assert could fire. Note that both fences are necessary: you need a release in one thread and an acquire in another to get a synchronizes-with relationship.